

PRE-CLASS READING 3 OF 4

The price is collapsing and the bill is rising — cost per successful task

A composite reading assembled for this class. Primary sources: Appenzeller, G. “Welcome to LLMflation”, a16z, 12 November 2024 • Epoch AI, “LLM inference price trends” (12 March 2025) and “LLM responses are getting longer” (17 April 2025, updated 20 November 2025) • Bessemer Venture Partners, “The AI pricing and monetization playbook”, 10 February 2026 • Artificial Analysis Intelligence Index v4.1 cost-per-task methodology, 15 June 2026. This is the one brief in the Week-5 set that the OBTL+ reading list does not name. It is here because **Capsule 2 asks you to build a cost-to-serve calculator**, and none of the assigned sources teaches the arithmetic you will need.

ASSIGNED READING

Read “Welcome to LLMflation” (≈ 8 min — short, one chart, one claim). Then Epoch AI’s “LLM responses to benchmark questions are getting longer over time” (≈ 5 min — read the chart, not the methodology). Then Bessemer’s AI pricing playbook (≈ 10 min). It is a *pricing* document, and we use it for *cost* on purpose: its central argument is **price on value, but you must know what you cost to serve** — and the cost list it gives to support that is the broadest anyone has published. Optional if you want the rigorous version: **Artificial Analysis’ cost-per-task methodology**. Read the first two together and hold one question the whole way: **if the price of a token has fallen a thousandfold, why is my AI bill going up?**

» sources ~23 min • this summary + reasoning notes ~26 min

Note on currency. Every price in this brief is a **spot price as of August 2026** and at least one of them will be wrong by the time you read it — Google launched Gemini 3.7 Flash on **13 August 2026** with a 50% introductory discount running to 31 December 2026, which means one model’s list price is scheduled to double before your next trimester begins. That is not a defect in the brief, it is the single most important design lesson in it: **prices belong in a config file, never in your code**, and any calculator that hardcodes them is obsolete before it ships. The Capsule-2 notebook is built that way deliberately.

KEY TAKEAWAY

Two forces are pulling in opposite directions and most people only know about one of them. **Force one: the price of a token is collapsing.** For a *fixed level of capability*, a16z measured the cost falling roughly **10× per year** — GPT-3-quality output went from **\$60 per million tokens in November 2021 to about \$0.06** three years later, a **1,000× fall**. Stanford’s AI Index (2025 edition) put the same effect at **280×** between November 2022 and October 2024 for GPT-3.5-level performance on MMLU: **\$20.00 down to \$0.07 per million tokens**. **Force two: the number of tokens a task consumes is exploding.** Epoch AI measures reasoning models emitting roughly **8× more tokens** than non-reasoning models on the same questions, with **output length** — the number of tokens the model *writes* in reply, which for a reasoning model includes the thinking tokens you are billed for but never see — growing **5× per year**. Anthropic reports that **agents use about 4× the tokens of a chat interaction, and multi-agent systems about 15×**. Multiply the two forces together and the honest conclusion is that **the unit price fell and your invoice grew**, because you moved from one cheap call to a loop of expensive ones. The consequence for anyone selecting or defending a use case is a change of unit. **Dollars per million tokens is not the number that decides anything. Cost per successful task is** — and as the worked example below shows, the two can rank your options in opposite orders.

“Will the price of a token go to zero?” — a question from a previous cohort, and worth answering properly, because it sounds like it settles the argument. **No** — and far more importantly, **it would settle very little if it did**. Take the worked example below and set the token price to **zero**. The cheap-agent option falls from **\$2.705 to \$2.700** per resolved ticket — a saving of **0.2%**. The frontier option falls from **\$1.485 to \$1.200**, about **19%**. **Free tokens do not change which architecture you should build**, because the human fallback still dominates both. That is this entire brief in one calculation: the question is about the smallest term in the equation. As for the price itself, three things hold it up. **(1)** The fall of roughly 10× a year is measured at **fixed capability**; the price of the *current frontier* is sticky, because the frontier is by definition what is only just affordable. **(2)** Every provider needs a gross margin, and AI-native margins run **50–60%** against 80–90% for classic software — so today’s promotional rates are not a steady state, and one of the prices quoted in this brief expires on **31 December 2026**. Prices have gone up as well as down. **(3)** Underneath both sits a real physical floor: energy per token, memory bandwidth, and the amortised capital cost of the accelerators and data centres — currently being financed like infrastructure rather than like software. **Small, still falling, genuinely non-zero — and not the number that decides anything.**

WHY IT MATTERS FOR THIS COURSE

This reading is the arithmetic under everything else in Week 5. Reading 1 claims that in an AI operating model the value of scale never stops climbing; that claim is true of the fixed components and **false of the per-call ones**, and this is where you find out by how much. Reading 2 finds that only 39% of firms can see any AI impact at EBIT; a meaningful share of the missing value is being eaten between the demo and the income statement by costs nobody modelled. In **Capsule 2** you will build the calculator that makes this concrete for your own use case, extending Week 3’s token economics and Week 4’s agent economics into a decision tool you can put in front of a sponsor. And for **A2**

and your **project**, this is the section of your write-up that separates a student exercise from a proposal: not “we built an agent”, but “it costs **X** cents per resolved case against **Y** cents for the human path, here is the sensitivity, and here is the success rate at which the answer flips.”

Feeds Class 5: C1 (which use cases are viable at all) • C2 (the cost-to-serve calculator you build — this is its specification) • C3 (why Ant’s pennies-per-transaction cost base was the strategy, not a detail).

MAIN POINTS

Force one — the price of a fixed capability collapses about 10× a year

- **The a16z measurement.** Hold *performance* constant and watch price: an MMLU score of 42 cost **\$60 per million tokens** on GPT-3 in November 2021 and about **\$0.06** on Llama 3.2 3B three years later. Roughly **10× per year, 1,000× over three years** — faster than Moore’s law plus Dennard scaling managed during the PC era. (*Both are explained in the appendix, along with why that era’s free lunch is not available to this one.*)
- **The Stanford AI Index measurement,** on a different benchmark: querying a model at **GPT-3.5 level (64.8 on MMLU)** fell from **\$20.00 to \$0.07 per million tokens** between November 2022 and October 2024 — **over 280×** — the Index says “approximately 18 months”, though those two dates are 23 months apart, which is a small illustration of Reading 2’s point about checking numbers you are handed.
- **Epoch AI’s caution, and it matters.** The rate depends entirely on which capability you hold fixed: measured across benchmarks, the price to reach a given score falls anywhere from **9× to 900× per year**. “10× a year” is a useful planning heuristic and a bad forecast.
- **Six things drive it (a16z):** better GPU cost-performance; quantisation from 16-bit toward 4-bit; software and serving optimisations; **smaller models beating much larger older ones**; better instruction tuning; and open-weight competition compressing margins through the stack. Note that only the first is hardware — this is mostly a software story, which is why it has been faster than chip progress.
- **What this actually buys you.** Not a smaller bill. It buys **viability**: use cases that were absurd at \$20 per million tokens are routine at \$0.10, and the frontier of what is worth automating moves down-market every year. Reading 1’s point about serving previously unservable segments is this curve, expressed as strategy.

Force two — the number of tokens per task explodes

- **Reasoning models emit about 8× more tokens** than non-reasoning models on the same benchmark questions (Epoch AI, 77 models, 235 evaluations). Output length is growing about **5× per year for reasoning models** against 2.2× for non-reasoning ones. Moving an OpenAI model from medium to high reasoning effort alone produced a **1.6× token increase**.
- **Agents multiply it again.** Anthropic’s own figure: **agents ≈ 4× the tokens of a chat interaction; multi-agent systems ≈ 15×**. Week 4 gave you the mechanism — the whole conversation is resent every turn, so input tokens grow with roughly the **square** of the number of turns.
- **Energy tells the same story from another angle.** Jegham et al. (“How Hungry is AI?”, arXiv:2505.09598) measure a short **GPT-4o** query at about **0.42 Wh**, against roughly **17.15 Wh for GPT-5 at high reasoning effort** on a medium prompt — about **40×** for the same question asked of a thinking model. Cost, energy and latency all move together, because they are all measuring the same thing: tokens generated.
- **The net effect.** Price per token down ~10× a year; tokens per task up several-fold a year, plus a step change whenever you move from a call to a loop. For a fixed feature your bill falls. **For an ambitious feature it rises**, and the ambition is what you actually bought.

Worked example — one use case, four architectures, and a ranking that flips

A support-triage assistant handling **50,000 tickets a month**. Prices are August 2026 spot: a cheap tier at **\$0.10 in / \$0.40 out** per million tokens, a frontier tier at **\$5 in / \$25 out**. Work it through slowly — this is exactly the calculation Capsule 2 asks you to automate.

- **v1 • single call.** 1,500 input + 400 output tokens. Cost = $(1,500 \div 1M \times \$0.10) + (400 \div 1M \times \$0.40) = \$0.00015 + \$0.00016 = \mathbf{\$0.00031 \text{ per ticket}}$, or about **\$15.50 a month**. At this price the token bill is not a business consideration at all.
- **v2 • add retrieval.** Four thousand tokens of retrieved context per ticket, so input becomes 5,500. Cost = $\$0.00055 + \$0.00016 = \mathbf{\$0.00071 \text{ per ticket}}$, about **\$35.50 a month**, plus roughly **\$50 a month** of fixed embedding and vector-store cost. Still trivial, and note that the *fixed* line is now larger than the variable one.
- **v3 • make it an agent, six turns, cheap model.** The API is **stateless** — nothing is remembered between calls — so the whole conversation is re-sent as input every turn. Each completed turn adds about **800 tokens** to it: roughly **400 of the model’s own reply** (its thought and chosen action) plus about **400 of observation** (what the tool returned). Input over six turns = $(6 \times 5,500) + 800 \times (0+1+2+3+4+5) = 33,000 + 12,000 = \mathbf{45,000 \text{ tokens}}$; output = $6 \times 400 = \mathbf{2,400}$. Cost = $\$0.0045 + \$0.00096 = \mathbf{\$0.00546 \text{ per ticket}}$, about **\$273 a month**. That is **17.6×** v1 (and 7.7× v2) — and nothing about the model changed. (*Notice what the model’s reply costs you: once at the*

output price on the turn it is written, and again at the input price on every later turn. That second charge is already inside the 800 — there is nothing further to add for it.)

- **v4 · same agent, frontier model.** Same 45,000 input and 2,400 output tokens at \$5 / \$25: $\$0.225 + \$0.06 = \$0.285$ per ticket, about **\$14,250 a month**. That is $52 \times v3$ and $920 \times v1$, for one feature.

On cost per token, the ranking is obvious: $v1 < v2 < v3 < v4$. Now add the only column that matters.

Suppose **v3 resolves 55% of tickets** unaided and **v4 resolves 80%**, and every unresolved ticket falls back to a human who spends eight minutes on it at a fully-loaded **UUS\$45 an hour — \$6.00 per escalation**. (Keep one currency throughout: the token prices are US dollars, so the wage is too.) Cost per ticket becomes token cost plus expected human cost. **v3: $\$0.00546 + (0.45 \times \$6.00) = \$2.71$. v4: $\$0.285 + (0.20 \times \$6.00) = \$1.49$.** The architecture that costs **52× more per token is 45% cheaper per resolved ticket**, and choosing on token price would have cost you about **\$61,000 a month — roughly \$730,000 a year** — on a 50,000-ticket queue. (Prices and the wage are both in US dollars; mixing currencies here is the single most common way this calculation goes wrong.)

Three things fall out of that, and they are the three things to bring to class. **First, the ranking inverted** — cost per token and cost per successful task are different orderings, and only one of them is a business number. **Second, the token bill was never the dominant cost.** The human fallback is **99.8% of cost per ticket in v3 and 81% in v4**; in both, a five-point improvement in resolution rate is worth more than a 90% cut in token price. Optimising the visible line while ignoring the dominant one is the most common mistake in this whole subject. **Third, you can solve for the break-even.** Set v3's cost equal to v4's: $\$0.00546 + (1 - p) \times \$6.00 = \$1.485$, giving **$p \approx 75.3\%$** . So the cheap agent wins only if you can push its resolution rate from 55% to about 75% — which is now a *concrete engineering target* rather than an argument, and it is the kind of number that ends a meeting.

Why AI gross margins are 50–60% and not 80–90%

- **The structural fact.** Bessemer's **pricing playbook** (February 2026) — it opens on margins because you cannot price what you have not costed: AI-native companies run **50–60% gross margins against 80–90% for classic software**. Their explanation: *"Unlike traditional SaaS, where serving one more customer costs virtually nothing, every AI query incurs a non-trivial expense."* Inference is **cost of goods sold** — a variable cost that scales with usage — where classic software's COGS was hosting and rounded to nothing.
- **And the harder a company pushes for growth, the thinner the margin gets.** Bessemer's separate *State of AI 2025* benchmarking (August 2025) sorts AI companies into growth archetypes by how fast they reach **\$100m of ARR — annual recurring revenue, the yearly value of subscriptions currently in force**. Its **"Supernovas"** get there in about eighteen months (roughly **\$40m of ARR in year one, \$125m in year two**) and run gross margins of about **25%**, with a footnote saying that for these companies margins are *often negative*. Its **"Shooting Stars"** take the slower route — about \$3m in year one, \$100m over four years — and run about **60%**. Same industry, same models available to both: **the difference is how hard they are pushing**, and growth that fast is bought by serving demand at a price that does not yet cover the inference behind it. ICONIQ, surveying around 300 executives building AI products, expects about **52% on average in 2026** — improving, and still nowhere near software's 80–90%.
- **The pricing consequence, and note that Bessemer's advice is the opposite of the obvious one.** Their observation is that founders *default* to cost-plus — calculate delivery cost, double it, and systematically undercharge — and their argument is to price on **value** instead, precisely because a cost-linked price hands the customer all the upside of every efficiency you find. What you must still do is **know** the cost to serve, and their list is broader than tokens: **model inference, human-in-the-loop support, customer-success overhead, and sales time**. Their test is worth memorising: **"if the math doesn't work at 10 customers, it won't at 1,000."** Variable costs do not amortise.
- **Which is why outcome pricing appeared — and you have already watched it happen.** When cost of goods sold scales with usage, **per-seat pricing breaks: the heaviest user becomes the least profitable customer**, and with a flat fee and no cap the tail stops being a rounding error and becomes an open-ended liability. **Class 3** gave you the shape: usage is heavily skewed, a small minority of users generating most of the consumption. **Class 4** watched Google act on it — Gemini Spark went from **US\$249.99 a year, to US\$100 at I/O in May 2026, to the US\$20 Pro tier by 27 July**, and its tiers differ by **how much work the agent may do (roughly 5× and 20× the Pro allowance), not by features**. Cost scales with steps taken; revenue would have scaled with seats sold. Hence that week's conclusion — **a usage cap is not a nicety, it is the only thing between a flat fee and an unbounded liability** — and the number to watch is cost per user per month at the **95th percentile**, not the average. **Outcome pricing is the other answer to the same problem:** charge per unit of value delivered, so revenue moves with the thing that drives cost. Intercom's Fin at **\$0.99 per resolved ticket** is the canonical example — and notice it is priced in exactly the unit this reading argues for.

The knobs — what actually reduces cost, in order of leverage

- **Raise the success rate.** Almost always the largest lever, and it is worth saying plainly *why*: **every failure means a person has to finish the job, and human time is by far the most expensive input in the system**. In the worked example one escalation costs **\$6.00** — eight minutes of somebody at a fully-loaded UUS\$45 an hour — against

\$0.00546 for the entire six-turn agent run that failed to prevent it. **One escalation costs roughly 1,100× the agent attempt that failed.** That is why the **expected cost of escalation**, not the token bill, is **99.8% of cost per resolved ticket in v3 and 81% in v4** — and why five points of resolution rate are worth more than a 90% cut in token price. Everything from Weeks 3 and 4 — better prompts, evals, tools, gates — is a **cost-reduction programme wearing a quality-improvement label.**

- **Route by difficulty.** Send the easy majority to a cheap model and escalate only hard cases to a frontier one. ICONIQ reports that many companies building AI products now route the **majority of their workloads** to smaller or fine-tuned models, escalating to a frontier model only for high-complexity cases. The 40–50× price spread between tiers is what makes it worth the engineering.
- **Cache aggressively — and be precise about what it fixes.** Cached input typically costs about **10% of fresh input** (OpenAI cached input is one tenth of list; Anthropic cache reads are 0.1× input, with writes at 1.25× for five minutes or 2× for an hour). A conversation is **append-only**, so everything before the newest increment is a byte-identical prefix and can be served from cache. The full-price bill therefore collapses to about **$B + D \times (T-1)$** — the base once, plus each turn's new increment — which is **linear**, while the whole re-read is billed at a tenth. On our six-turn run that is **9,500 tokens at full price against 35,500 cache-read: a 71% saving on input.** At twenty turns it is **83%**, because the share being re-read grows with the loop.
- **Batch anything not interactive.** A flat **50% discount** across all three major vendors. If a job can wait, waiting is worth half the bill.
- **Control the observation size — and keep the two terms in proportion.** Week 4's lesson, priced: what a tool returns goes into every subsequent turn, so a tool returning 200 tokens instead of 2,000 shrinks **D**, and **D** drives the growing term. But look at the whole formula before you optimise. $\text{Input} = B \times T$ (the base re-sent every turn — linear) + $D \times T(T-1)/2$ (the conversation re-read — quadratic). With our numbers the quadratic half is only **27% of a six-turn run**, and it does not overtake the linear half until about **fifteen turns** ($T > 2B/D + 1$). So on short loops the bigger prize is **shrinking B** — the system prompt, the tool descriptions, the retrieved context you send every single turn — while **observation size is what decides whether a long loop is affordable at all.** Cap steps either way: an unbounded loop is an unbounded invoice.
- **Move work out of the model entirely.** Anything deterministic — validation, arithmetic, lookups, formatting — belongs in code, where it costs nothing and cannot hallucinate. This is Week 4's "guarantees belong in code" argument, restated as an economics argument, and it is the same argument.

MORE TOKENS IS NOT MORE ACCURACY — THE EVIDENCE

The tempting inference from all of the above is that spending more buys more. The best available evidence says the relationship is not monotonic, and that is worth knowing before you write a business case that assumes it is.

- **The Holistic Agent Leaderboard** (Kapoor, Narayanan, Liang et al., arXiv 2510.11977, October 2025) ran **21,730 agent rollouts** across 9 models × 9 benchmarks, spending about **\$40,000** and logging **2.5 billion tokens**. Headline finding: **higher reasoning effort reduced accuracy in the majority of runs.** More thinking, more tokens, worse answers — often enough to be the default expectation rather than a curiosity.
- **The cost-accuracy inversion, concretely.** On Princeton's HAL leaderboard for SWE-bench Verified: **Claude 3.5 Sonnet scored 38.0% at \$67.09 total run cost**, while **o1-mini scored 27.2% at \$366.81 — 5.5× the cost for 10.8 points less accuracy.** (These are 2024-vintage models; the lesson is the shape, not the numbers.)
- **Which is why the reporting unit changed.** Artificial Analysis' Intelligence Index moved in June 2026 to reporting **Cost per Task, Time per Task and Tokens per Task**, explicitly to reflect the expense of agentic rather than chat workloads. Across the whole index, cost per task runs from about **\$0.04 to \$3.25** and time per task from **1.5 to 13.5 minutes**. The comparison that matters is the like-for-like one: at broadly similar index positions they report systems **20× to 45× apart on cost** — for example **\$0.99** against **\$1.78** at the top, and far cheaper options within a point or two of them.
- **The practical rule.** Evaluate on a **cost-accuracy frontier**, not on accuracy alone. A configuration is only worth defending if nothing cheaper matches it. "We used the best model" is not an engineering decision; it is the absence of one.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to give you a direction to push on; the fuller reasoning is in the section after.

- Token prices fell about 1,000× in three years and enterprise AI bills went up. Explain the mechanism precisely enough that someone could predict which of their own use cases will get cheaper and which will get more expensive.

Clue. Two variables, not one — write the bill as **price per token × tokens per task × tasks**, and ask which of the three you actually control. Then notice that the falling term is measured at fixed capability, and that nobody buys

fixed capability: they spend the saving on a harder task. The prediction you want falls out of asking whether a given use case is holding its ambition constant.

- In the worked example the token cost is **under 1%** of the cost per resolved ticket for v3 and about **19%** for v4 — the smaller half in both. What does that imply about where engineering effort should go, and what would have to be true for the answer to change?

Clue. *Do the sensitivity rather than reasoning about it: recompute cost per resolved ticket with the token price halved, then again with the resolution rate raised five points, and compare the two savings. Then ask what would make the token line dominant — think about volume, about whether a human is in the fallback path at all, and about what happens when the fallback is another model instead of a person.*

- Higher reasoning effort reduced accuracy in the majority of runs in a 21,730-rollout study. How can spending more tokens make an answer worse?

Clue. *Resist treating this as noise. Start from what extra reasoning tokens physically are — more generated text, sitting in the context, conditioning everything that follows. Connect that to Week 4's context rot and to why a reviewer with a clean context finds bugs the author cannot. Then ask what kinds of task have a short correct path, and what a model does when given a long budget for one of those.*

- Your cheap architecture resolves 55% of cases and your expensive one resolves 80%. Break-even is about 75%. Which do you ship on Monday, and what do you do next?

Clue. *This is a sequencing question, not a preference question — notice that shipping one does not prevent improving the other, and that only one of the two options generates the data you would need to improve either. Think about what a wrong answer costs in each case, how quickly you would find out, and what the 25% gap in resolution rate is actually made of before you assume it is closeable.*

- Write the cost-to-serve formula you will implement in Capsule 2. Name every term, including the ones that are not tokens.

Clue. *Build it in three layers and keep them separate: **per-task variable** (input, cached input, output, retrieval, tool calls), **per-task expected fallback** (probability of failure × the cost of handling a failure), and **fixed monthly** (vector store, infrastructure, evaluation, the engineer maintaining it). Bessemer's list is broader than yours will be — check it and see what you left out. Then make every price a named parameter, because at least one of them changes before December.*

HOW TO THINK ABOUT THESE — read after you have had a go

These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so they are written to stand on their own.

Q1 — The falling price is measured at fixed capability, and nobody buys fixed capability

Write the bill as **price per token × tokens per task × number of tasks**. The famous 1,000× applies only to the first term, and only when you hold the *output quality* constant — a16z's comparison is explicitly an MMLU-42 model in 2021 against an MMLU-42 model in 2024. But essentially nobody responds to a price fall by continuing to buy the 2021 capability. They spend the windfall on the second term: they turn a single call into a retrieval-augmented call, then into a six-turn agent, then into a reasoning model that emits eight times the tokens. Every one of those is a deliberate purchase of a harder task, and the invoice reflects the purchase, not a pricing failure. Jevons' paradox, if you want the economics label — when a resource gets cheaper, total consumption rises because uses that were uneconomic become economic. The prediction rule falls straight out and it is genuinely useful. **A use case whose ambition is frozen gets cheaper every year, quickly.** Batch classification, extraction, summarisation of a fixed corpus — if the spec does not change, the line item falls by roughly an order of magnitude a year, and you should plan budgets on that basis rather than extrapolating today's cost. **A use case whose ambition tracks the frontier gets more expensive**, because the frontier is defined by what is only just affordable. Three that behave this way. **Coding agents** — as models improve nobody keeps them on the same small tickets; they get pointed at bigger repositories and harder bugs. **Multi-document analysis** — “summarise this report” quietly becomes “read forty of them, reconcile the figures and flag the contradictions”, a different task carrying the same name. **And the ladder in this brief** — v1 to v4 is four successive decisions to spend the saving on a harder version of the same job. And there is a corollary worth stating to a sponsor before they hear it elsewhere: the falling price is not a reason to wait. Waiting buys you a cheaper version of a capability you also did not have last year, while the compounding assets — the pipeline, the eval set, the workflow integration — only accumulate if you start.

Q2 — Optimise the dominant term, and know what would make it move

The sensitivity settles it. Halving the token price takes v4 from \$1.485 to about \$1.343 per resolved ticket — a saving of roughly **\$0.14**, or 10%. Raising the resolution rate by five points, from 80% to 85%, takes it to about \$1.185 — a saving of roughly **\$0.30**, more than twice as much, for a change most teams could achieve with better prompts and a better eval set. So on this shape of problem the engineering priority is unambiguous: **work on the failure rate, not the token bill**, and treat model-cost optimisation as housekeeping rather than strategy. It is worth naming why

teams do the opposite. The token bill is visible — it arrives monthly, with a chart, addressed to someone who has to explain it — while the escalation cost is buried in an operations budget that predates the project and is nobody's AI line item. Visibility, not magnitude, drives attention. Three conditions would flip the answer, and each is worth recognising. **Volume without a human fallback:** a consumer feature serving ten million requests a day where a failure just means a mediocre answer has no \$6.00 escalation, so tokens are close to 100% of marginal cost. **Very high success rates:** at 97% resolution the residual human cost is \$0.18 and the token line becomes comparable. **A model-based fallback:** if failures escalate to a frontier model rather than a person, the fallback is tokens too, and the whole calculation collapses into one currency. Diagnosing which regime you are in is the first thing your Capsule-2 calculator should tell you, and it is why the formula must carry the fallback term explicitly rather than hiding it in an assumption.

Q3 — Long thinking on a short problem is a way of talking yourself out of the answer

The mechanism is the same one Week 4 gave you for context rot, applied to the model's own output. Reasoning tokens are generated text that then sits in the context and conditions everything after it, and the model is built to continue text coherently rather than to notice that it has gone wrong. On a problem whose correct path is short, a long reasoning budget is an invitation to construct elaborations, consider and reject the right answer, and commit to a plausible chain that has drifted — and once the chain is written down, everything downstream is conditioned on it. A second effect is worth separating from the first: more effort also expands the *space of things attempted*, so on agentic benchmarks a longer budget means more tool calls, more chances at an irreversible wrong action, and more compounding. HAL's result — higher effort reducing accuracy in the majority of runs, across 9 models and 9 benchmarks — is strong enough that "more reasoning" should be treated as a **tunable parameter with an optimum**, not a quality dial that only goes up. The operational consequence is concrete and cheap: **reasoning effort belongs in your eval sweep alongside temperature and model choice** — the appendix says how to set it, and why it is not the same kind of dial. Run your own eval set at low, medium and high effort and look at the accuracy-versus-cost frontier; the middle setting often wins on both axes, which is a rare and satisfying result. And notice how neatly this closes the loop with Week 3: the reason you built an eval harness in the first place was so that questions like this could be answered by measurement in an afternoon instead of by seniority in a meeting.

Q4 — Ship the cheap one, instrument it, and find out what the 25% is made of

Ship v3 on Monday, with three conditions attached. **Log every escalation with its cause**, because the 25% resolution gap is not a homogeneous mass — it is some mixture of cases needing knowledge the system lacks, cases needing a tool it does not have, cases needing judgement it should not be making, and cases it got wrong for no good reason. Those four have completely different remedies and only one of them is "buy a better model". **Run v4 in shadow mode on a sample** — same inputs, no action taken — so you accumulate a real comparison on your own traffic instead of arguing from the vendor's benchmark, which Week 4 taught you is a lower bound on capability and silent about usefulness. **Set a review date and a kill condition** in advance. The reason this sequencing is right rather than merely cautious is that shipping v3 is the only option that generates the data needed to improve *either* option, while v4 costs 52× more per token from day one to buy an advantage you have not yet verified on your traffic. There is also a real chance the gap closes for free: if a meaningful share of escalations are missing-tool or missing-context cases, then a better tool or a better retrieval step fixes them on the cheap model, and 55% moves toward 75% without a model upgrade. That would be the ideal outcome and it is not a rare one. Two honest caveats to hold alongside all of this. If a wrong answer is expensive or irreversible — money moving, a clinical judgement, anything a regulator reads — the 55% option is not on the table at any price, and the calculation is not a cost calculation at all. And the 75.3% break-even is only as good as the two success rates that produced it; both are estimates, and the first thing your Capsule-2 calculator should do after computing a break-even is show how much it moves when those estimates are wrong by five points.

Q5 — The formula, in three layers

Keep the layers separate, because each behaves differently as you scale. **Layer 1, per-task variable:** (fresh input tokens × input price) + (cached input tokens × cached price, typically 0.1× input) + (output tokens × output price) + (retrieval or embedding cost) + (external tool or API call fees). For an agent, remember that input is the accumulating conversation, so it grows with roughly the square of turns — model it as a sum over turns, not as one multiplication, or you will understate it badly. **Layer 2, per-task expected fallback:** $(1 - \text{success rate}) \times \text{cost of handling a failure}$, where that cost is human minutes at a loaded rate, or a frontier-model retry, or in some businesses the cost of the error itself. This layer is usually the largest and is the one most often omitted, which is precisely why so many pilots look profitable and so few deployments are. **Layer 3, fixed monthly:** vector store and infrastructure, evaluation runs, monitoring, and the fraction of an engineer who keeps it alive. Bessemer would add two more that you will be tempted to leave out and shouldn't if you are pricing a product rather than costing a feature: **customer-success overhead and sales time**, both of which are real costs of serving a customer. Then two disciplines that turn a formula into a tool. **Every price is a named parameter read from configuration** — the Gemini 3.7 introductory discount expires on 31 December 2026 and your calculator should survive that without an edit. And **report cost per successful task, not cost per call**, with a sensitivity table over the success rate, because the success rate is simultaneously the term you are least sure of and the one the answer is most sensitive to. Build that, and you have something a sponsor can act on rather than a number they have to trust.

APPENDIX — UNIT-ECONOMICS VOCABULARY, EXPLAINED

The distinction that carries this reading: **price per token is an input price; cost per successful task is a business cost.**

Everything below exists to keep the two apart, because almost every bad AI investment decision comes from optimising the first.

Cost to serve

- **Plain.** Everything it costs you to deliver the service to one customer, once.

Technically. Per-task variable cost + expected failure-handling cost + an allocated share of fixed cost. Bessemer's components: model inference, human-in-the-loop support, customer-success overhead, sales time. Their test: *if the math doesn't work at 10 customers, it won't at 1,000* — because variable costs do not amortise.

Cost per successful task

- **Plain.** What you pay for each outcome you actually wanted, including the ones you had to redo.

Technically. Per-task token and tool cost **plus** $(1 - \text{success rate}) \times \text{the cost of handling a failure}$ — the three-layer formula in Q5, and the one the Capsule-2 notebook implements. Adopted as a reporting standard by Artificial Analysis in June 2026 (Cost per Task / Time per Task / Tokens per Task) precisely because per-token pricing stopped predicting agentic workload expense.

LLMflation

- **Plain.** For the same quality of answer, the price falls about tenfold every year.

Technically. a16z, November 2024. MMLU-42 capability: **\$60/M tokens (Nov 2021) → \$0.06/M (2024)**, ~1,000× in three years. Measured at **fixed capability** — which is the caveat that makes it compatible with rising bills. Epoch AI's cross-benchmark range is 9×–900× per year.

Moore's law and Dennard scaling — and why the PC era's free lunch is not on the menu

- **Plain.** Two separate things made computers get better AND cheaper AND cooler for thirty years. Only one of them is still running.

Technically. **Moore's law** (Gordon Moore, 1965, rev. 1975) is an observation about *economics*, not physics: transistors per chip at the lowest cost per transistor roughly **double every two years**. It gives you more transistors for your money. **Dennard scaling** (Dennard et al., IEEE JSSC, 1974) is the physics that let you use them: shrink a transistor's dimensions and its operating voltage together and **power density stays constant**. Together, a chip could hold more transistors *and* run them faster *and* not get hotter — clock speeds rose about **40% every two years**. **Dennard scaling broke between about 2005 and 2007**, around 65 nm: voltage stopped scaling because leakage grew exponentially. Clocks stalled near 3–4 GHz, the industry pivoted to **more cores**, and “dark silicon” appeared. **Why it belongs here:** the honest objection to this brief is “*compute became effectively free once, so why not tokens?*” — and the answer is that the PC era ran on **two** engines and the AI era has one. Today's gains come from more silicon, costlier packaging and memory, lower precision and much more power (~700 W for a data-centre GPU against 65–100 W for a PC processor): performance bought with capital and electricity, not handed over by physics. And even then, users' bills did not fall — software expanded to consume every gain. Force two of this brief, eighty years early.

Fixed vs variable cost, restated for AI

- **Plain.** What you pay once, versus what you pay again every time somebody uses it.

Technically. Inference is **cost of goods sold** — variable, linear in usage. Pipelines, eval harnesses and infrastructure are fixed and amortise. Reading 1's “scale never stops paying” applies to the fixed bucket only; this is the precise sense in which it aged.

Gross margin

- **Plain.** Of every dollar of revenue, how much is left after the cost of delivering the thing.

Technically. $(\text{Revenue} - \text{COGS}) \div \text{revenue}$. AI-native companies run **50–60%** against **80–90%** for classic software (Bessemer, Feb 2026); the fast-growth cohort nearer **25%**, sometimes negative; ICONIQ's ~300 AI builders expect about **52% in 2026**. The gap is inference.

Prompt caching

- **Plain.** Pay full price for the long unchanging preamble once, then a tenth of the price to reuse it.

Technically. Cached input is typically **0.1× the input price**; Anthropic charges **1.25× input** to write a 5-minute cache and **2× input** for a 1-hour cache. In an agent loop with a large stable system prompt, this is the single biggest structural saving available.

Batch inference

- **Plain.** If it can wait a few hours, it costs half.

Technically. A flat **50% discount** across OpenAI, Anthropic and Google as of August 2026. Applies to anything non-interactive — enrichment, classification, back-filling, evaluation runs. Frequently the cheapest optimisation available and the last one anyone implements.

Model routing / escalation

- **Plain.** Cheap model for the easy majority, expensive model only for the hard ones.
Technically. ICONIQ reports that many firms building AI products route the majority of their **workloads** this way. Worth the engineering because the price spread between the cheapest credible tier and the frontier is roughly **40–50× on input** — and because the routing decision itself can usually be made by a cheap model or by code.

Reasoning effort — and why it is not another temperature

- **Plain.** A dial for how long the model thinks before it answers. Unlike temperature, moving it changes your bill.
Technically. **Temperature** changes how randomly the next token is chosen: it costs nothing and does not change how many tokens you buy. **Reasoning effort** changes how many tokens the model generates *before* it starts answering, and those thinking tokens are **billed as output** — so it is a direct cost and latency dial, and you are usually paying for text you never see. Two idioms: **categorical** — OpenAI's ``reasoning_effort`` (minimal / low / medium / high), Gemini 3's ``thinkingLevel`` — and a **token budget**, as in Anthropic's extended thinking ``budget_tokens`` (minimum 1,024). **On OpenRouter, which is what you hold a key for, both are unified** into one ``reasoning`` object: pass either ``effort`` or ``max_tokens``, plus ``exclude: true`` if you do not want the reasoning text back. It converts between them — roughly high = 80% of the budget, medium = 50%, low = 20%. **Two cautions.** The dials are not independent: some providers constrain temperature when extended thinking is on (Anthropic pins it to 1), so you cannot freely cross them in a sweep. And parameter names move every few months — **the two idioms do not**, so learn the shapes, not the spellings.

Cost-accuracy frontier

- **Plain.** Plot every configuration by what it costs and how often it is right; only the ones nothing beats on both are worth arguing about.
Technically. The evaluation standard argued for by the Holistic Agent Leaderboard (arXiv 2510.11977, Oct 2025; 21,730 rollouts). Accuracy-only reporting hides inversions like 38.0% at \$67 beating 27.2% at \$367.

Outcome pricing

- **Plain.** Charge per result delivered rather than per user or per message.
Technically. The natural response to usage-linked COGS, since per-seat pricing makes your heaviest user your least profitable one. Canonical example: **Intercom Fin at \$0.99 per resolved ticket** — priced in exactly the unit this reading argues you should be measuring.

SOURCE LINKS

Companion readings in this pack, referred to by number throughout: **Reading 1** Iansiti & Lakhani, *Competing in the Age of AI* — the “marginal cost approaches zero” claim this brief prices • **Reading 2** McKinsey, *The State of AI* • **Reading 4** GraphCast, *Climate TRACE and AI in verticals* • **Case pre-read 1** Ant Group (A).

a16z — Guido Appenzeller, “Welcome to LLMflation” (12 Nov 2024) • a16z.com/llmflation-llm-inference-cost/ (the 10×-per-year claim)

Epoch AI — “LLM responses to benchmark questions are getting longer over time” • epoch.ai/data-insights/output-length (the 8× reasoning-token figure)

Epoch AI — “LLM inference price trends” (12 Mar 2025) • epoch.ai/data-insights/llm-inference-price-trends (the 9×–900× range)

Bessemer Venture Partners — “The AI pricing and monetization playbook” (10 Feb 2026) • www.bvp.com/atlas/the-ai-pricing-and-monetization-playbook (a PRICING document — we use it for the gross-margin figures and its cost-to-serve checklist, because its argument is “price on value, but know your cost”. Also the best pricing reference in the set.)

Artificial Analysis — Intelligence Index v4.1, cost per task (15 Jun 2026) • artificialanalysis.ai/articles/artificial-analysis-intelligence-index-v4-1

Kapoor, Narayanan, Liang et al. — Holistic Agent Leaderboard (arXiv 2510.11977, Oct 2025) • arxiv.org/abs/2510.11977 (more reasoning effort, worse accuracy)

Anthropic — “How we built our multi-agent research system” (13 Jun 2025) • www.anthropic.com/engineering/built-multi-agent-research-system (agents ≈4×, multi-agent ≈15× the tokens of a chat)

Stanford HAI — AI Index 2025, Chapter 1 (the 280× inference-cost figure) • hai.stanford.edu/assets/files/hai_ai-index-report-2025_chapter1_final.pdf

Jegham et al. — “How Hungry is AI?” (arXiv:2505.09598) • arxiv.org/abs/2505.09598 (the energy-per-query figures)

Bessemer — State of AI 2025 benchmarking slides (Aug 2025) • www.bvp.com/assets/uploads/2025/08/Final_PDF_State_of_AI_2025_slides_Bessemer_Venture_Partners.pdf (a different Bessemer document — benchmarking slides; source of the Supernova / Shooting Star 25% and 60% archetype margins)

Companion • **Reading 1** claims scale pays forever — this brief says which half of the cost structure that is true of; **Reading 2** finds the value missing at EBIT, and some of it is being eaten here.

CAVEATS

1. Every price quoted is a **spot price as of August 2026** taken from vendor pricing pages. At least one will have changed before you read this — that is the point of the note on currency, not a defect. Check the vendor page before you use a number in an assignment.
2. The worked example is illustrative. The token counts, the resolution rates and the US\$45/hour loaded cost are plausible placeholders chosen to make the arithmetic legible, not measurements of any real support queue. The *structure* is what transfers; substitute your own numbers.
3. The HAL SWE-bench cost figures involve 2024-vintage models and should not be read as current frontier economics. They are included because the inversion is the clearest published illustration of the point.
4. Bessemer's 50–60% vs 80–90% margin figure is a general industry observation rather than a named sample. ICONIQ's ~52% ($n \approx 300$) and Bessemer's own 25% / 60% archetype benchmarks are the better-sourced numbers; all three are self-reported by companies with an interest in the answer.
5. A composite assembled by the instructor, not a summary of one source, and the only reading this week the OBTL+ outline does not name. The three-layer formula, the worked example and the break-even are the instructor's; the figures are the cited sources'. Generated and fact-checked using Anthropic models — a vendor whose pricing appears in it. Flag anything that looks wrong.